

Problem 5.13.16

Sarvesh Tamgade

September 20, 2025

Question: Let k be an integer such that the triangle with vertices

$$\mathbf{A} = \begin{pmatrix} k \\ -3k \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 5 \\ k \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} -k \\ 2 \end{pmatrix}$$

has area 28 sq. units. Find the orthocenter of the triangle.

$$\text{Area} = \frac{1}{2} |\mathbf{M}| = 28 \quad (2.1)$$

$$\text{where } \mathbf{M} = (\mathbf{B} - \mathbf{A} \quad \mathbf{C} - \mathbf{A}) = \begin{pmatrix} 5 - k & -2k \\ 4k & 2 + 3k \end{pmatrix} \quad (2.2)$$

$$\det(\mathbf{M}) = (5 - k)(2 + 3k) - (-2k)(4k) = 10 + 13k + 5k^2 \quad (2.3)$$

$$10 + 13k + 5k^2 = \pm 56 \quad (2.4)$$

After solving,

$$k = 2 \quad (2.5)$$

Solution

$$\text{For } k = 2, \quad \mathbf{A} = \begin{pmatrix} 2 \\ -6 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 5 \\ 2 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} -2 \\ 2 \end{pmatrix} \quad (2.6)$$

The equation of the altitude through $\mathbf{A}$ is:

$$\mathbf{x}^T \mathbf{n} = 1 \quad (2.7)$$

This altitude passes through $\mathbf{A}$, so

$$\mathbf{A}^T \mathbf{n} = 1 \quad (2.8)$$

$$\begin{pmatrix} 2 & -6 \end{pmatrix} \mathbf{n} = 1 \quad (2.9)$$

and is perpendicular to $\mathbf{B} - \mathbf{C}$, so

$$(\mathbf{B} - \mathbf{C})^T \mathbf{n} = 0 \quad (2.10)$$

$$\begin{pmatrix} 7 & 0 \end{pmatrix} \mathbf{n} = 0 \quad (2.11)$$

Solution

The augmented matrix is:

$$\left(\begin{array}{cc|c} 7 & 0 & 0 \\ 2 & -6 & 1 \end{array} \right) \quad (2.12)$$

Row reduction steps:

First, $R_2 \rightarrow R_2 - \frac{2}{7}R_1$:

$$\left(\begin{array}{cc|c} 7 & 0 & 0 \\ 0 & -6 & 1 \end{array} \right) \quad (2.13)$$

$$\mathbf{n} = \begin{pmatrix} 0 \\ -\frac{1}{6} \end{pmatrix} \quad (2.14)$$

The equation of the altitude through $\mathbf{B}$ is:

$$\mathbf{x}^T \mathbf{n} = 1 \quad (2.15)$$

Solution

This altitude passes through $\mathbf{B}$, so

$$\mathbf{B}^T \mathbf{n} = 1 \quad (2.16)$$

$$\begin{pmatrix} 5 & 2 \end{pmatrix} \mathbf{n} = 1 \quad (2.17)$$

and is perpendicular to $\mathbf{A} - \mathbf{C}$, so

$$(\mathbf{A} - \mathbf{C})^T \mathbf{n} = 0 \quad (2.18)$$

$$\begin{pmatrix} 4 & -8 \end{pmatrix} \mathbf{n} = 0 \quad (2.19)$$

The augmented matrix is:

$$\left(\begin{array}{cc|c} 5 & 2 & 1 \\ 4 & -8 & 0 \end{array} \right) \quad (2.20)$$

Solution

Row reduction steps: First, $R_2 \rightarrow R_2 - \frac{4}{5}R_1$:

$$\left(\begin{array}{cc|c} 5 & 2 & 1 \\ 0 & -\frac{48}{5} & -\frac{4}{5} \end{array} \right) \quad (2.21)$$

Next, $R_2 \rightarrow -\frac{5}{48}R_2$:

$$\left(\begin{array}{cc|c} 5 & 2 & 1 \\ 0 & 1 & \frac{1}{12} \end{array} \right) \quad (2.22)$$

Back substitute: $R_1 \rightarrow R_1 - 2R_2$

$$\left(\begin{array}{cc|c} 5 & 0 & \frac{5}{6} \\ 0 & 1 & \frac{1}{12} \end{array} \right) \quad (2.23)$$

$$\mathbf{n} = \begin{pmatrix} \frac{1}{6} \\ \frac{1}{12} \end{pmatrix} \quad (2.24)$$

Solution

First altitude:

$$\begin{pmatrix} 0 & -\frac{1}{6} \end{pmatrix} \mathbf{x} = 1 \quad (2.25)$$

Second altitude:

$$\begin{pmatrix} \frac{1}{6} & \frac{1}{12} \end{pmatrix} \mathbf{x} = 1 \quad (2.26)$$

Combining both equations:

$$\begin{pmatrix} 0 & -\frac{1}{6} \\ \frac{1}{6} & \frac{1}{12} \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (2.27)$$

Writing the augmented matrix:

$$\left(\begin{array}{cc|c} \frac{1}{6} & \frac{1}{12} & 1 \\ 0 & -\frac{1}{6} & 1 \end{array} \right) \quad (2.28)$$

Solution

Perform the row operation $R_1 \rightarrow R_1 + \frac{1}{2}R_2$:

$$\left(\begin{array}{cc|c} \frac{1}{6} & 0 & \frac{3}{2} \\ 0 & -\frac{1}{6} & 1 \end{array} \right) \quad (2.29)$$

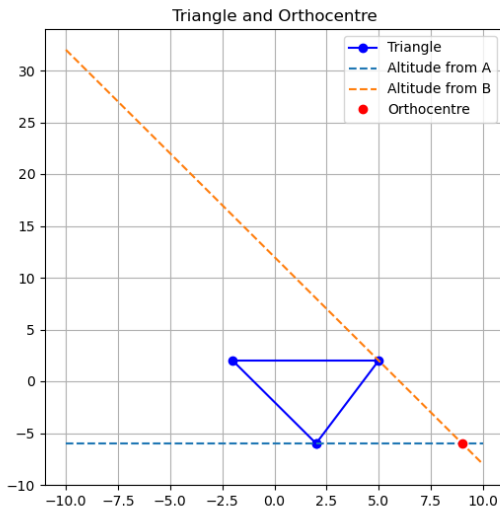
Therefore, the solution vector is:

$$\mathbf{x} = \begin{pmatrix} 9 \\ -6 \end{pmatrix} \quad (2.30)$$

Answer: The orthocentre of the triangle is

$$\boxed{\mathbf{x} = \begin{pmatrix} 9 \\ -6 \end{pmatrix}}$$

Graph



```
#include "matfun.h"

void line_equation(const double P[2], const double Q[2], const
    double R[2], double *a, double *b, double *c) {
    double dx = R[0] - Q[0];
    double dy = R[1] - Q[1];
    *a = -dy;
    *b = dx;
    *c = (*a)*P[0] + (*b)*P[1];
}

void solve_2x2(double a1, double b1, double c1, double a2, double
    b2, double c2, double sol[2]) {
    double det = a1*b2 - a2*b1;
    sol[0] = (b2*c1 - b1*c2) / det;
    sol[1] = (a1*c2 - a2*c1) / det;
}
```

```
void compute_orthocentre(const double A[2], const double B[2],  
    const double C[2], double O[2]) {  
    double a1, b1, c1, a2, b2, c2;  
    line_equation(A, B, C, &a1, &b1, &c1);  
    line_equation(B, A, C, &a2, &b2, &c2);  
    solve_2x2(a1, b1, c1, a2, b2, c2, O);  
}
```

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt

lib = ctypes.CDLL('./matfun.so')

arr_type = ctypes.c_double * 2
A = arr_type(2, -6)
B = arr_type(5, 2)
C = arr_type(-2, 2)
O = arr_type()

lib.compute_orthocentre.argtypes = [arr_type, arr_type, arr_type,
                                     arr_type]
lib.line_equation.argtypes = [arr_type, arr_type, arr_type,
                              ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.c_double)]
```

```
lib.solve_2x2.argtypes = [ctypes.c_double, ctypes.c_double,  
    ctypes.c_double, ctypes.c_double, ctypes.c_double, ctypes.  
    c_double, arr_type]  
  
# Call orthocentre computation  
lib.compute_orthocentre(A, B, C, 0)  
print("Orthocentre:", list(0))  
  
# Arrays to hold line coefficients  
a1, b1, c1 = ctypes.c_double(), ctypes.c_double(), ctypes.  
    c_double()  
a2, b2, c2 = ctypes.c_double(), ctypes.c_double(), ctypes.  
    c_double()  
a3, b3, c3 = ctypes.c_double(), ctypes.c_double(), ctypes.  
    c_double()
```

```
# Compute altitude lines via C functions for all three vertices
lib.line_equation(A, B, C, ctypes.byref(a1), ctypes.byref(b1),
    ctypes.byref(c1)) # Altitude from A
lib.line_equation(B, A, C, ctypes.byref(a2), ctypes.byref(b2),
    ctypes.byref(c2)) # Altitude from B
lib.line_equation(C, A, B, ctypes.byref(a3), ctypes.byref(b3),
    ctypes.byref(c3)) # Altitude from C

# Convert points to numpy for plotting
A_np = np.array([2, -6])
B_np = np.array([5, 2])
C_np = np.array([-2, 2])
O_np = np.array([0[0], 0[1]])

fig, ax = plt.subplots(figsize=(6,6))
ax.plot([A_np[0], B_np[0], C_np[0], A_np[0]], [A_np[1], B_np[1],
    C_np[1], A_np[1]], 'bo-', label='Triangle')
```

```
def plot_line(a, b, c, ax, color, label):
    if abs(b) > 1e-8:
        x = np.linspace(-10, 20, 300)
        y = (c - a*x)/b
    else:
        x = np.full(300, c/a)
        y = np.linspace(-10, 20, 300)
    ax.plot(x, y, color=color, linestyle='--', label=label)

plot_line(a1.value, b1.value, c1.value, ax, 'r', 'Altitude from A')
plot_line(a2.value, b2.value, c2.value, ax, 'g', 'Altitude from B')
plot_line(a3.value, b3.value, c3.value, ax, 'b', 'Altitude from C')
```



```
ax.plot(O_np[0], O_np[1], 'ro', label='Orthocentre')  
  
ax.legend()  
ax.grid(True)  
ax.set_title('Triangle, Altitudes, and Orthocentre (using .so  
functions)')  
plt.show()
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

A = np.array([2, -6])
B = np.array([5, 2])
C = np.array([-2, 2])

def altitude_through_point(P, Q, R):
    """
    Returns coefficients a,b,c of the altitude through P
    which is perpendicular to QR, i.e.  $a*x + b*y = c$ 
    """
    # Direction vector of QR
    d = Q - R
    # Normal vector for the altitude (perpendicular)
    n = np.array([-d[1], d[0]])
    c = n[0]*P[0] + n[1]*P[1]
    return n[0], n[1], c
```

Python Code

```
def intersect_lines(a1, b1, c1, a2, b2, c2):  
    """  
    Finds the intersection point of two lines:  $a_1x + b_1y = c_1$  and  
     $a_2x + b_2y = c_2$   
    """  
    A = np.array([[a1, b1], [a2, b2]])  
    b = np.array([c1, c2])  
    return np.linalg.solve(A, b)  
  
a1, b1, c1 = altitude_through_point(A, B, C) # through A, perp to  
BC  
a2, b2, c2 = altitude_through_point(B, A, C) # through B, perp to  
AC  
  
O = intersect_lines(a1, b1, c1, a2, b2, c2)  
  
print("Orthocentre:", O)
```

```
# Plotting
plt.figure(figsize=(6,6))
plt.plot([A[0], B[0], C[0], A[0]], [A[1], B[1], C[1], A[1]], 'bo-
', label='Triangle')

# Plot altitudes
def plot_altitude(P, n, c, label):
    # Generate points along the altitude for plot
    t = np.linspace(-10, 10, 100)
    if n[1] != 0:
        x = t
        y = (c - n[0]*x)/n[1]
    else:
        x = np.ones_like(t)*(c/n[0])
        y = t
    plt.plot(x, y, '--', label=label)
```

```
plot_altitude(A, np.array([a1, b1]), c1, 'Altitude from A')
plot_altitude(B, np.array([a2, b2]), c2, 'Altitude from B')

plt.plot(O[0], O[1], 'ro', label='Orthocentre')
plt.legend()
plt.grid(True)
plt.title('Triangle and Orthocentre')
plt.savefig("fig1.png")
plt.show()
```